



DRF yordamida REST API yaratish. CRUD API ishlab chiqish

Generic view'lar (ListCreateAPIView, RetrieveUpdateDestroyAPIView), ViewSet va Router, ruxsatlar (faqat muallif), sahifalash, qidiruv va saralash, token autentifikatsiya, CORS; Front-end (JavaScript fetch) va Back-end integratsiyasi; mahsulotlar API'si.

2 soat · amaliy

Bu darsda nimani o'rganasiz

- To'liq CRUD API'ni (GET, POST, PUT, PATCH, DELETE) bir necha qatorda yozishni;
- **ViewSet** va **Router** — URL'larni avtomatik yaratishni;
- API ruxsatlari: "faqat muallif o'zgartira oladi";
- Sahifalash, qidiruv va saralashni;
- **Front-end va Back-end integratsiyasi**: II bobdagi JavaScript bilan Django API'dan foydalanishni (29-loyiha);
- Mahsulotlar boshqaruv API'sini (27–28-loyihalar).

Abstraksiya zinapoyasi

39-mavzuda `@api_view` bilan har bir metodni qo'lda yozdik. DRF bir xil ishni tobora qisqaroq yozish imkonini beradi:

Daraja	Vosita	Kod hajmi	Qachon
1	<code>@api_view</code> / <code>APIView</code>	Ko'p, hammasi qo'lda	Nostandart mantiq

2	Generic view'lar (<code>ListCreateAPIView</code> ...)	Kam	Standart CRUD, URL'larni o'zingiz boshqarasiz
3	<code>ModelViewSet</code> + <code>Router</code>	Eng kam	To'liq standart CRUD



O'xshatish: ovqat tayyorlash

1-daraja — hamma narsani noldan pishirish. 2-daraja — yarim tayyor mahsulotlardan. 3-daraja — tayyor taomni isitib berish. Har birining o'z o'rni bor: maxsus taom uchun noldan pishirasiz, oddiy tushlik uchun — tayyoridan foydalanasiz.

2-daraja: Generic view'lar

blog/api_views.py

```
from rest_framework import generics
from .models import Maqola
from .serializers import MaqolaSerializer

class MaqolaListCreate(generics.ListCreateAPIView):          # GET (ro'yxat) +
    queryset =                                               POST (yaratish)
    queryset =
    Maqola.objects.filter(chop_etilgan=True).select_related("kategoriya",
    "muallif")
    serializer_class = MaqolaSerializer

    def perform_create(self, serializer):                    # saqlashdan oldin
    muallifni qo'shamiz
        serializer.save(muallif=self.request.user)

class MaqolaDetail(generics.RetrieveUpdateDestroyAPIView):  # GET (bitta) +
    queryset = Maqola.objects.all()                          PUT + PATCH + DELETE
    serializer_class = MaqolaSerializer
```

blog/urls.py

```
path("api/maqolalar/", api_views.MaqolaListCreate.as_view()),
path("api/maqolalar/<int:pk>/", api_views.MaqolaDetail.as_view()),    #
generic'lar "pk" kutadi
```

Atigi ~15 qatorda to'liq CRUD API! Ammo hozir **istalgan** tizimga kirgan foydalanuvchi boshqa birovning maqolasini o'zgartira oladi — buni tuzatamiz.

Ruxsatlar (permissions)

blog/permissions.py (yangi fayl)

```
from rest_framework import permissions

class MuallifYokiFaqatOqish(permissions.BasePermission):
    """O'qish – hammaga; o'zgartirish va o'chirish – faqat muallifga."""

    def has_object_permission(self, request, view, obj):
        if request.method in permissions.SAFE_METHODS:    # GET, HEAD,
OPTIONS – "xavfsiz" metodlar
            return True
        return obj.muallif == request.user
```

blog/api_views.py

```
from rest_framework import permissions
from .permissions import MuallifYokiFaqatOqish

class MaqolaDetail(generics.RetrieveUpdateDestroyAPIView):
    queryset = Maqola.objects.all()
    serializer_class = MaqolaSerializer
    permission_classes = [permissions.IsAuthenticatedOrReadOnly,
MuallifYokiFaqatOqish]
```

Tayyor ruxsat	Ma'nosi
<code>AllowAny</code>	Hammaga ochiq
<code>IsAuthenticated</code>	Faqat tizimga kirganlar
<code>IsAuthenticatedOrReadOnly</code>	O'qish — hammaga, yozish — kirganlarga
<code>IsAdminUser</code>	Faqat admin (staff)

Bu 38-mavzudagi "faqat muallif tahrirlay oladi" qoidasining API'dagi ko'rinishi. Begona foydalanuvchi PUT/DELETE yuborsa — `403 Forbidden`.

3-daraja: ViewSet va Router

blog/api_views.py

```
from rest_framework import viewsets, permissions, filters
from .models import Maqola, Kategoriya
from .serializers import MaqolaSerializer, KategoriyaSerializer
from .permissions import MuallifYokiFaqatOqish

class MaqolaViewSet(viewsets.ModelViewSet):          # list, create, retrieve,
update, partial_update, destroy
    serializer_class = MaqolaSerializer
    permission_classes = [permissions.IsAuthenticatedOrReadOnly,
MuallifYokiFaqatOqish]
    filter_backends = [filters.SearchFilter, filters.OrderingFilter]
    search_fields = ["sarlavha", "matn"]              # ?search=futbol
    ordering_fields = ["yaratilgan", "korishlar"]      # ?ordering=-
korishlar

    def get_queryset(self):
        qs =
Maqola.objects.filter(chop_etilgan=True).select_related("kategoriya",
"muallif")
        kategoriya = self.request.query_params.get("kategoriya")    # ?
kategoriya=sport
        if kategoriya:
            qs = qs.filter(kategoriya__slug=kategoriya)
        return qs

    def perform_create(self, serializer):
        serializer.save(muallif=self.request.user)

class KategoriyaViewSet(viewsets.ReadOnlyModelViewSet):    # faqat o'qish: list
+ retrieve
    queryset = Kategoriya.objects.all()
    serializer_class = KategoriyaSerializer
```

config/urls.py

```
from rest_framework.routers import DefaultRouter
from blog.api_views import MaqolaViewSet, KategoriyaViewSet

router = DefaultRouter()
router.register("maqolalar", MaqolaViewSet, basename="maqola")
router.register("kategoriyalar", KategoriyaViewSet)

urlpatterns = [
    path("admin/", admin.site.urls),
    path("api/", include(router.urls)),          # ← barcha API URL'lari
    # avtomatik
    path("api-auth/", include("rest_framework.urls")), # brauzerdagi API'da
    # "Log in" tugmasi
    path("", include("blog.urls")),
]
```



39-mavzu va yuqoridagi generic misol uchun `blog/urls.py` ga qo'shgan `api/maqolalar/...` yo'llarini endi **o'chiring** — aks holda bir xil manzil ikki joyda e'lon qilinadi.

Router avtomatik yaratgan URL'lar:

URL	Metodlar	ViewSet metodi
<code>/api/</code>	GET	Barcha endpointlar ro'yxati (DRF bosh sahifasi)
<code>/api/maqolalar/</code>	GET, POST	<code>list</code> , <code>create</code>
<code>/api/maqolalar/5/</code>	GET, PUT, PATCH, DELETE	<code>retrieve</code> , <code>update</code> , <code>partial_update</code> , <code>destroy</code>
<code>/api/kategoriyalar/</code> , <code>/api/kategoriyalar/2/</code>	GET	<code>list</code> , <code>retrieve</code>

Sahifalash

config/settings.py

```
REST_FRAMEWORK = {
    "DEFAULT_PERMISSION_CLASSES":
    ["rest_framework.permissions.IsAuthenticatedOrReadOnly"],
    "DEFAULT_PAGINATION_CLASS":
    "rest_framework.pagination.PageNumberPagination",
    "PAGE_SIZE": 10,
}
```

GET /api/maqolalar/?page=2&search=maktab&ordering=-korishlar

```
{
  "count": 23,
  "next": "http://127.0.0.1:8000/api/maqolalar/?page=3&search=maktab&ordering=-korishlar",
  "previous": "http://127.0.0.1:8000/api/maqolalar/?page=1&search=maktab&ordering=-korishlar",
  "results": [
    { "id": 12, "sarlavha": "Maktab kutubxonasi yangilandi", "korishlar": 340,
    "...": "..." }
  ]
}
```

Mobil ilova va JavaScript uchun: Token autentifikatsiya

Brauzer sessiya (cookie) orqali tizimda qoladi. Mobil ilova va boshqa dasturlar esa har bir so'rovda **token** — maxfiy kalit yuboradi.

config/settings.py

```
INSTALLED_APPS += ["rest_framework.authtoken"]

REST_FRAMEWORK["DEFAULT_AUTHENTICATION_CLASSES"] = [
    "rest_framework.authentication.SessionAuthentication",
    "rest_framework.authentication.TokenAuthentication",
]
```

config/urls.py

```
from rest_framework.authtoken.views import obtain_auth_token
urlpatterns += [path("api/token/", obtain_auth_token)] # POST
username+password → token
```

Terminal

```
python manage.py migrate
curl -X POST http://127.0.0.1:8000/api/token/ -d
"username=ali&password=parol12345"
# {"token": "9944b09199c62bcf9418ad846dd0e4bbdfc6ee4b"}

curl -X POST http://127.0.0.1:8000/api/maqolalar/ \
-H "Authorization: Token 9944b09199c62bcf9418ad846dd0e4bbdfc6ee4b" \
-H "Content-Type: application/json" \
-d '{"sarlavha": "Token bilan yozildi", "matn": "Salom!", "kategoriya":
1}'
```



Token — parol kabi maxfiy! Uni kodga yozib GitHub'ga yuklamang. Haqiqiy loyihalarda tokenlar HTTPS orqali uzatiladi (41-mavzu).

Front-end va Back-end integratsiyasi (29-loyiha)

Endi kursning eng muhim lahzasi: **II bobdagi JavaScript** (front-end) va **Django API** (back-end) birlashadi. Front-end alohida sayt bo'lishi mumkin (masalan, Netlify'da), shuning uchun avval **CORS** ni sozlaymiz.

CORS — boshqa saytdan so'rovlarga ruxsat

Brauzer xavfsizlik uchun bir saytdagi JavaScript'ga **boshqa domendagi** API'ga so'rov yuborishni taqiqlaydi (**CORS** — Cross-Origin Resource Sharing). Ruxsat **server** tomonidan beriladi:

Terminal

```
pip install django-cors-headers
```

config/settings.py

```
INSTALLED_APPS += ["corsheaders"]

MIDDLEWARE = [
    "corsheaders.middleware.CorsMiddleware",          # ← ro'yxatning eng
    boshiga
    "django.middleware.security.SecurityMiddleware",
    # ... qolganlari
]

CORS_ALLOWED_ORIGINS = [
    "http://127.0.0.1:5500",                          # VS Code Live Server
    "https://maktab-blog.netlify.app",                 # internetdagi front-end
]
```

Front-end: maqolalar o'quvchi sahifa

frontend/index.html (Live Server bilan oching)

```
<!DOCTYPE html>
<html lang="uz">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Maktab blogi – JS mijoz</title>
<style>
  body { font-family: "Segoe UI", sans-serif; max-width: 760px; margin: 30px
auto; padding: 0 16px; background: #f8fafc; }
  .qidiruv { display: flex; gap: 8px; margin-bottom: 16px; }
  .qidiruv input { flex: 1; padding: 10px; border: 2px solid #e5e7eb; border-
radius: 10px; }
  .karta { background: #fff; padding: 16px 20px; border-radius: 14px; margin-
bottom: 12px; box-shadow: 0 4px 14px rgba(0,0,0,.06); }
  .belgi { background: #eef2ff; color: #4f46e5; padding: 2px 10px; border-
radius: 999px; font-size: 13px; }
  .xato { color: #dc2626; }
  button { padding: 10px 16px; border: none; border-radius: 10px; background:
#4f46e5; color: #fff; cursor: pointer; }
</style>
</head>
<body>
  <h1>📖 Maktab blogi</h1>
  <form class="qidiruv" id="qidiruv">
    <input id="soz" placeholder="Qidirish...">
    <button>Izlash</button>
  </form>
  <p id="holat">Yuklanmoqda... </p>
  <div id="royxat"></div>
  <button id="yana" hidden>Yana yuklash</button>

<script>
  const API = "http://127.0.0.1:8000/api/maqolalar/";
  const royxat = document.querySelector("#royxat");
  const holat = document.querySelector("#holat");
  const yanaBtn = document.querySelector("#yana");
  let keyingi = null;

  async function yukla(url, tozala = true) {
```

```

holat.textContent = "Yuklanmoqda...";
holat.className = "";
try {
  const javob = await fetch(url);
  if (!javob.ok) throw new Error(`Server xatosi: ${javob.status}`);
  const data = await javob.json(); // { count, next, previous,
results }

  if (tozala) royxat.innerHTML = "";
  data.results.forEach(m => {
    const karta = document.createElement("article");
    karta.className = "karta";
    const sarlavha = document.createElement("h3");
    sarlavha.textContent = m.sarlavha; // textContent
- XSS'dan himoya
    const meta = document.createElement("p");
    meta.innerHTML = `<span class="belgi"></span> 🖱️ <b></b> · 👁️
    ${m.korishlar}`;
    meta.querySelector(".belgi").textContent = m.kategoriya_nomi;
    meta.querySelector("b").textContent = m.muallif;
    const matn = document.createElement("p");
    matn.textContent = m.matn.slice(0, 150) + (m.matn.length > 150 ? "...": "");
    karta.append(sarlavha, meta, matn);
    royxat.append(karta);
  });

  holat.textContent = `Jami: ${data.count} ta maqola`;
  keyingi = data.next;
  yanaBtn.hidden = !keyingi;
} catch (err) {
  holat.textContent = `❌ ${err.message}. Django server ishlayaptimi?`;
  holat.className = "xato";
}
}

document.querySelector("#qidiruv").addEventListener("submit", e => {
  e.preventDefault();
  const soz = document.querySelector("#soz").value.trim();
  yukla(`${API}?search=${encodeURIComponent(soz)}`);
});

yanaBtn.addEventListener("click", () => yukla(keyingi, false)); // keyingi
sahifa

```

```
yukla(API);  
</script>  
</body>  
</html>
```

Nima bo'lyapti? JavaScript (front-end, port 5500) → `fetch` → Django API (back-end, port 8000) → PostgreSQL → JSON → JavaScript sahifani chizadi. Bu — **full-stack** ilovaning to'liq zanjiri, va siz uning barcha qismlarini o'zingiz yozdingiz! 🎉

 **index.html + JS**
Live Server :5500

— `fetch(GET /api/maqolalar/)` →

 **Django + DRF**
:8000

⇔ SQL ⇔  **PostgreSQL**

Amaliy mashg'ulot: mahsulotlar boshqaruv API'si (27–28-loyihalar)

1 Yangi ilova: `python manage.py startapp shop`. Model:

shop/models.py

```
from django.db import models  
  
class Mahsulot(models.Model):  
    nomi = models.CharField(max_length=200)  
    tavsif = models.TextField(blank=True)  
    narx = models.DecimalField(max_digits=12, decimal_places=2)  
    soni = models.PositiveIntegerField(default=0)  
    faol = models.BooleanField(default=True)  
    yaratilgan = models.DateTimeField(auto_now_add=True)  
  
    def __str__(self):  
        return self.nomi
```

- 2 `MahsulotSerializer` (narx musbat bo'lsin — `validate_narx`), `MahsulotViewSet` (qidiruv — nom; saralash — narx, yaratilgan), router'ga qo'shing.
- 3 Ruxsat: o'qish — hammaga, qo'shish/o'zgartirish/o'chirish — faqat admin (`IsAdminUser` ; maslahat: `get_permissions` metodida `self.action` ga qarab).
- 4 `?narx_min=10000&narx_max=50000` filtrini `get_queryset` da qo'shing.
- 5 Barcha amallarni brauzerdagi API sahifasida yoki Thunder Client'da sinang: GET (ro'yxat, bitta), POST, PATCH (faqat narx), DELETE.
- 6 **Front-end:** 19-mavzudagi kartalar dizayni bilan mahsulotlar katalogi sahifasini yarating — ma'lumotlar API'dan kelsin.

▼ 3-qadam uchun maslahat

Python

```
class MahsulotViewSet(viewsets.ModelViewSet):
    queryset = Mahsulot.objects.filter(faol=True)
    serializer_class = MahsulotSerializer

    def get_permissions(self):
        if self.action in ("list", "retrieve"):
            return [permissions.AllowAny()]
        return [permissions.IsAdminUser()]
```

Ko'p uchraydigan xatolar

- ✗ Brauzer konsolida `blocked by CORS policy` — `corsheaders` o'rnatilmagan, middleware ro'yxat boshida emas yoki front-end manzili `CORS_ALLOWED_ORIGINS` da yo'q (port ham mos bo'lishi kerak!).
- ✗ `ViewSet`'da `get_queryset` ishlatilganda router'da `basename` unutilgan.
- ✗ Generic view URL'ida `<int:id>` — generic'lar standart holatda `pk` kutadi.
- ✗ `fetch` natijasini `.json()` siz ishlatish yoki `await` ni unutish.

O'zingizni tekshiring

? Bitta klassda list, create, retrieve, update, destroy amallarini beruvchi DRF klassi?

- A. `APIView`
- B. `ListAPIView`
- C. `ModelViewSet`
- D. `Serializer`

? `ViewSet` uchun URL'larni avtomatik yaratuvchi vosita?

- A. Router (`DefaultRouter`)
- B. `path()`
- C. Paginator
- D. Middleware

? `SAFE_METHODS` qaysi metodlarni o'z ichiga oladi?

- A. POST, PUT
- B. DELETE
- C. Hammasi
- D. GET, HEAD, OPTIONS

? Netlify'dagi JS sayt Django API'ga so'rov yuborganda "CORS" xatosi chiqdi. Nima qilish kerak?

- A. JavaScript kodini o'chirish
- B. Django'da `django-cors-headers` ni sozlab, front-end manzilini ruxsat etilganlarga qo'shish
- C. Brauzerni almashtirish
- D. API'ni HTML'ga aylantirish

? API'da qidiruv uchun DRF'ning qaysi vositasi ishlatiladi?

- A. `Paginator`
- B. `OrderingFilter`
- C. `SearchFilter` + `search_fields`
- D. `Router`



Uyga vazifa

1. JS front-end'ga "Maqola qo'shish" formasini qo'shing: token bilan `POST` so'rov yuboring (`headers: { "Authorization": "Token ...", "Content-Type": "application/json" }`).
2. Kutubxona API'ngizni `ViewSet` + `Router`'ga o'tkazing, qidiruv va sahifalash qo'shing.



Yodda tuting

- Generic: `ListCreateAPIView`, `RetrieveUpdateDestroyAPIView`; `perform_create` — muallifni qo'shish.
- `ModelViewSet` + `DefaultRouter` — to'liq CRUD API bir necha qatorda.
- Ruxsatlar: `permission_classes`, o'z `BasePermission` klassingiz (`has_object_permission`).
- `SearchFilter`, `OrderingFilter`, `PageNumberPagination`; token autentifikatsiya.
- Integratsiya: front-end `fetch` ⇌ DRF API; `django-cors-headers` bilan CORS.

☐ Darsni o'qib bo'ldim

✓ Test javoblari

1. Bitta klassda list, create, retrieve, update, destroy amallarini beruvchi DRF klassi? — **C) ModelViewSet**
2. ViewSet uchun URL'larni avtomatik yaratuvchi vosita? — **A) Router (DefaultRouter)**
3. SAFE_METHODS qaysi metodlarni o'z ichiga oladi? — **D) GET, HEAD, OPTIONS**
4. Netlify'dagi JS sayt Django API'ga so'rov yuborganda "CORS" xatosi chiqdi. Nima qilish kerak? — **B) Django'da django-cors-headers ni sozlab, front-end manzilini ruxsat etilganlarga qo'shish**
5. API'da qidiruv uchun DRF'ning qaysi vositasi ishlatiladi? — **C) SearchFilter + search_fields**